Laboratory 5

Basics of Neural Network

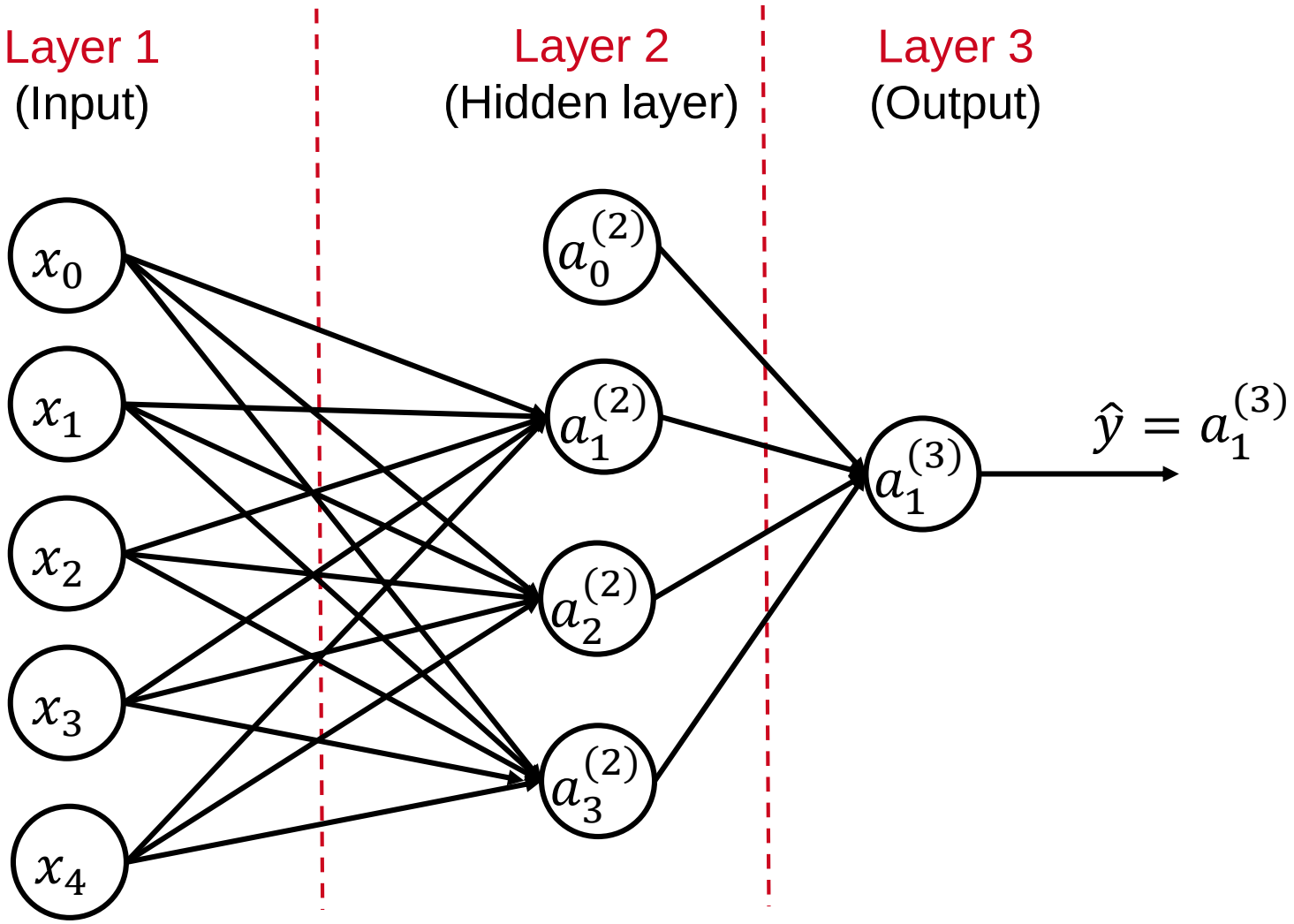
Sadaf Joodaki, Mohsen Pourghasemian.
emml@dsp.rwth-aachen.de

Overview of exercises

- Laboratory 1: K-Means Clustering (1.0 Point)
- Laboratory 2: Grid Search for Linear Regression (1.0 Point)
- Laboratory 3: Gradient Descent for Linear Regression (1.5 Point)
- Laboratory 4: Logistic Regression (1.0 Point)
- Laboratory 5: Basics of Neural Network (1.0 Point)
- Laboratory 6: Image Classification with Neural Network (1.5 Points)
- Laboratory 7: Reinforcement learning (1.5 Point)
- Laboratory 8: Deep RL (1.5 Point)

Fully connected network structure

- Fully connected neural network
  - One input layer
  - One or more hidden layers
  - One output layer

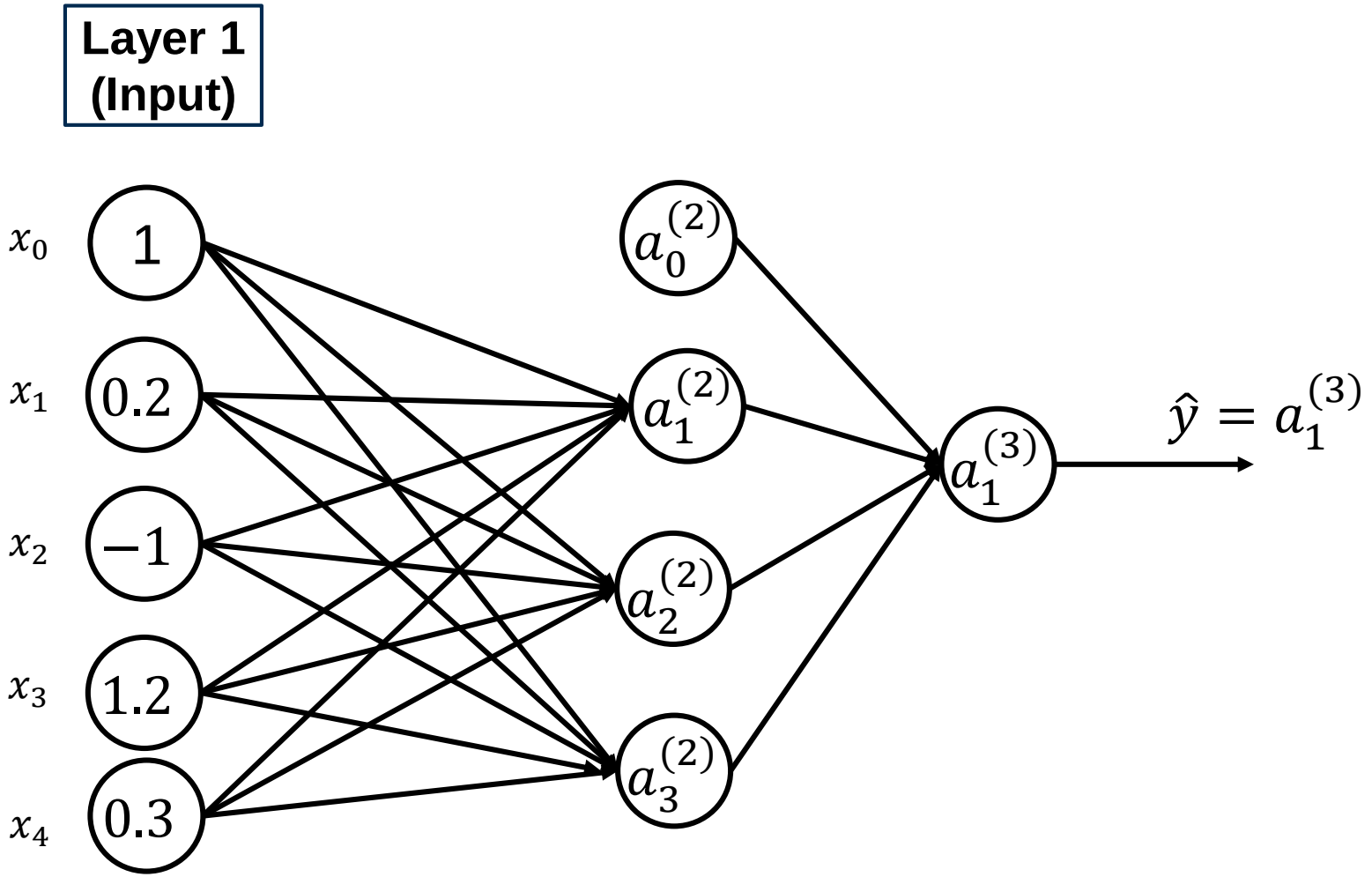


Forward Propagation

Input Layer

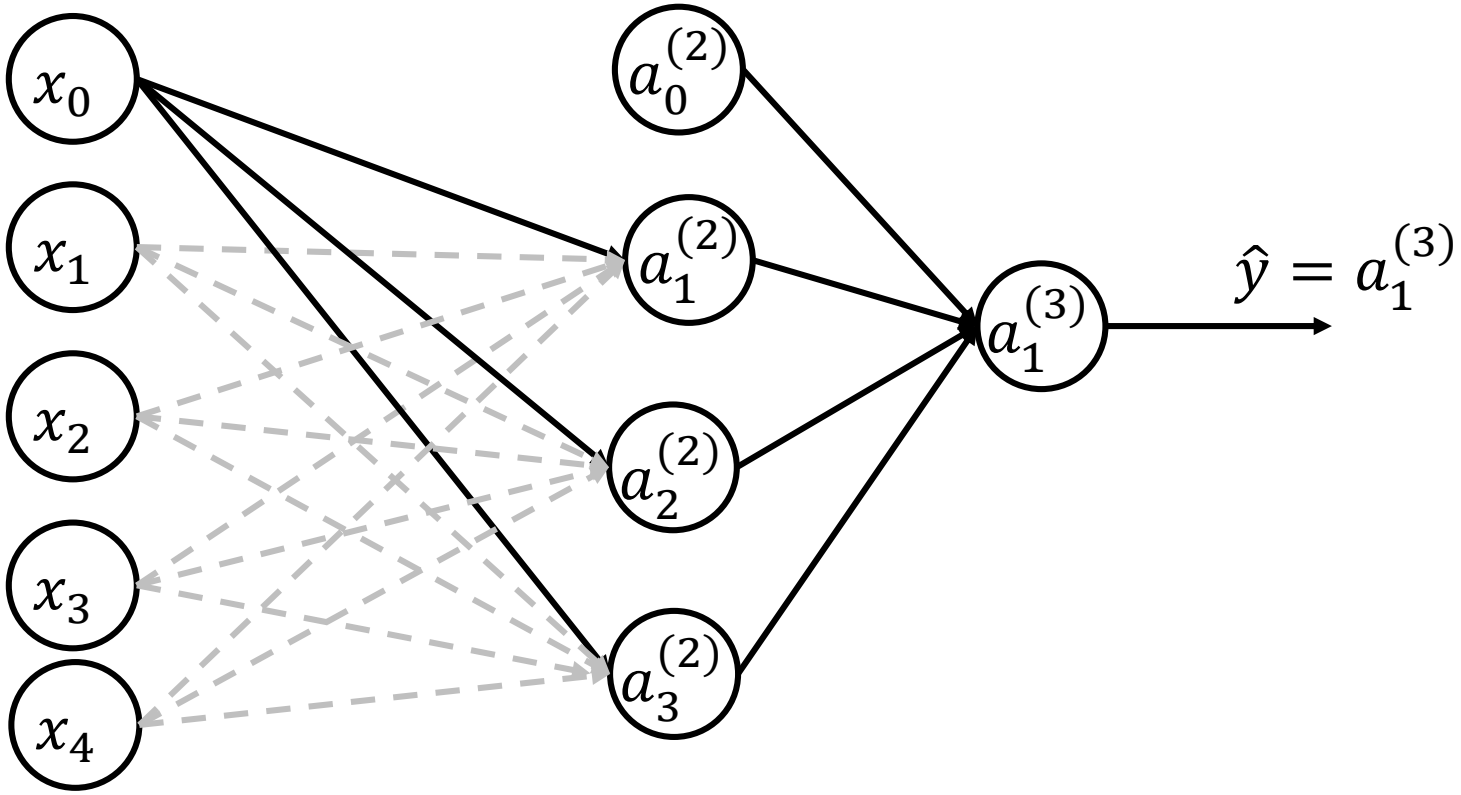
input	value
x_1	0.2
x_2	-1
x_3	1.2
x_4	0.3

- We assume $x_0 = 1$



Initial Weights

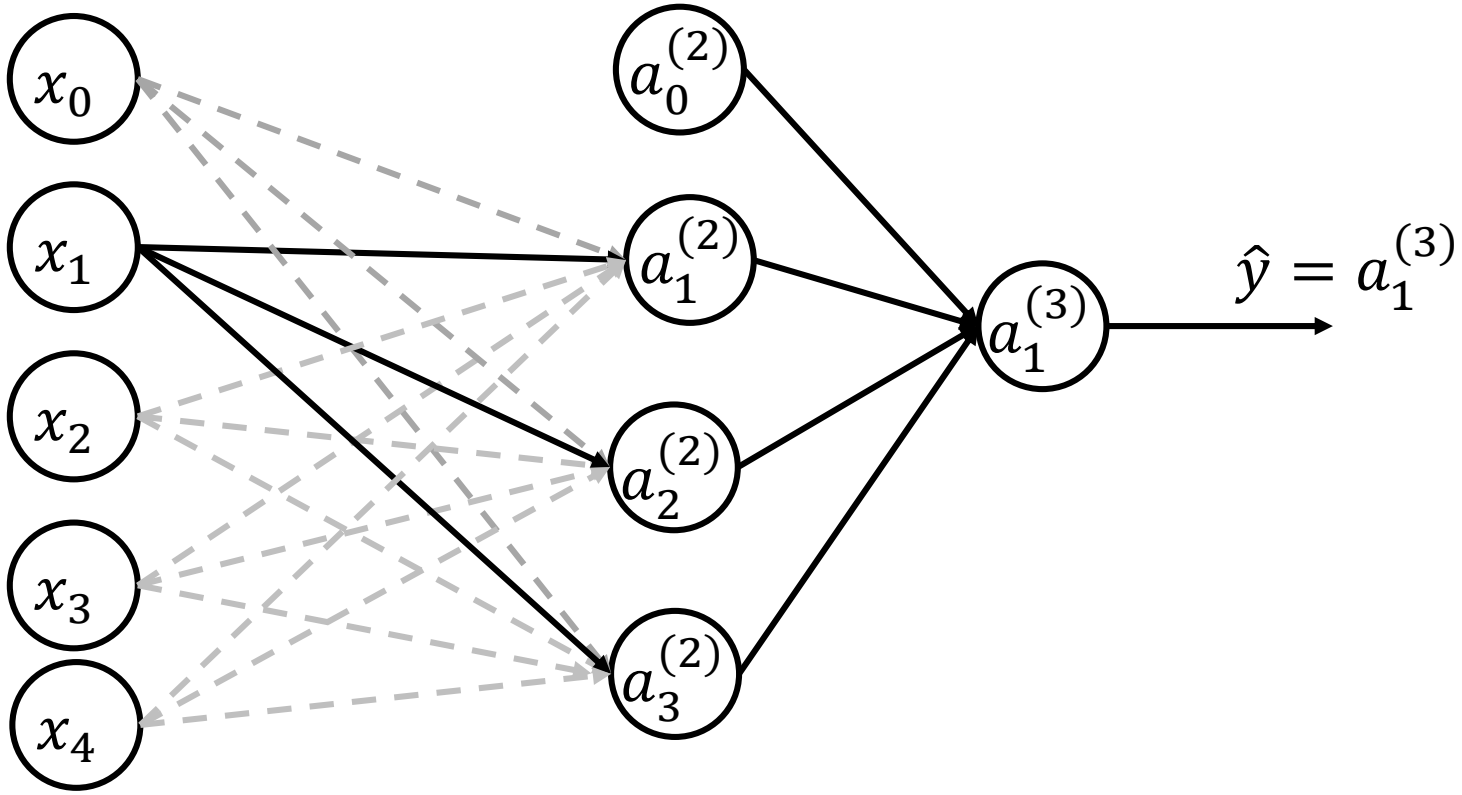
- The weights are initialized randomly and their values are illustrated in the table below.



Initial Weights	Value
$\theta_{10}^{(1)}$	1.0
$\theta_{20}^{(1)}$	0.3
$\theta_{30}^{(1)}$	0.4

Initial Weights

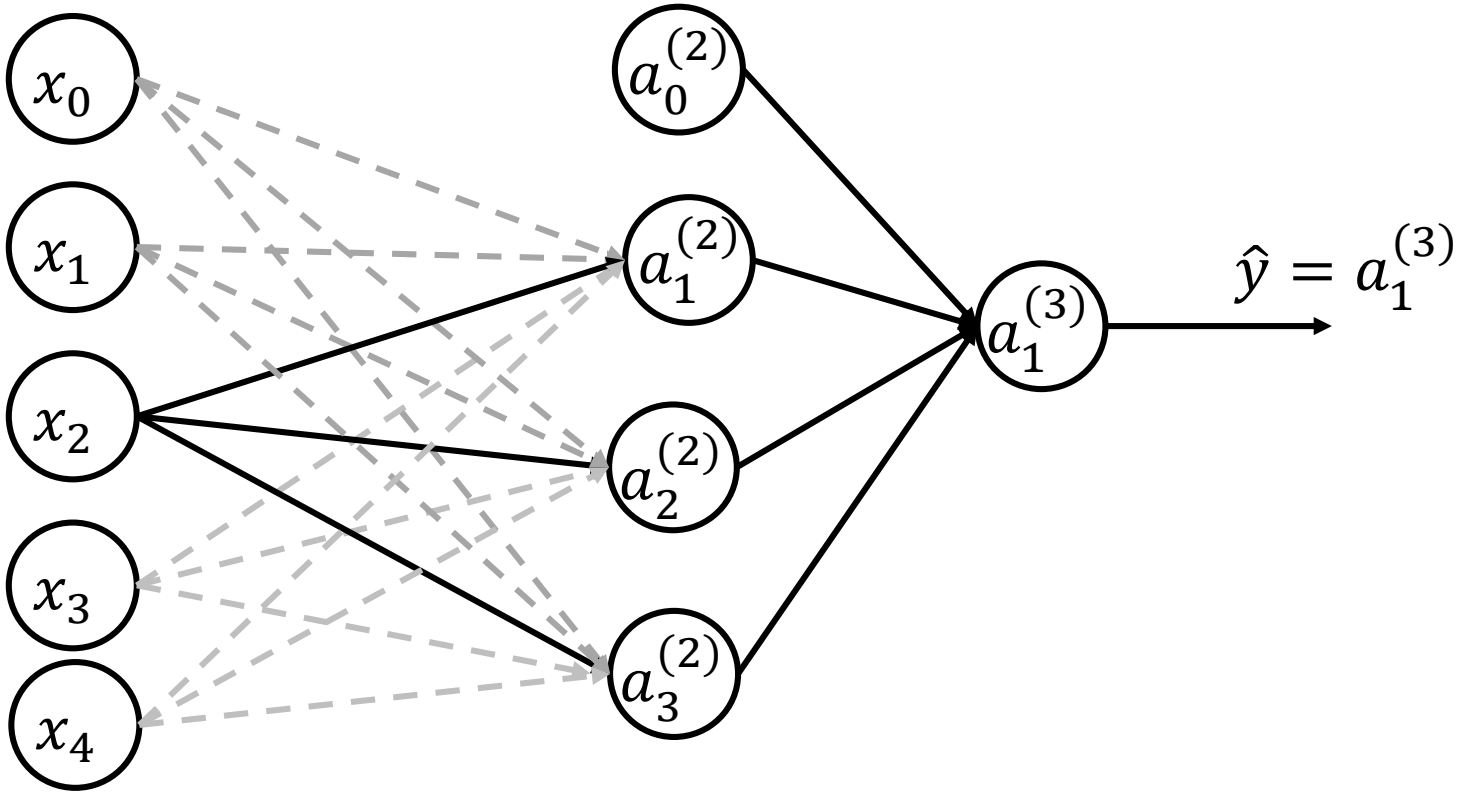
- The weights are initialized randomly and their values are illustrated in the table below.



Initial Weights	Value
$\theta_{11}^{(1)}$	0.2
$\theta_{21}^{(1)}$	1.2
$\theta_{31}^{(1)}$	0.1

Initial Weights

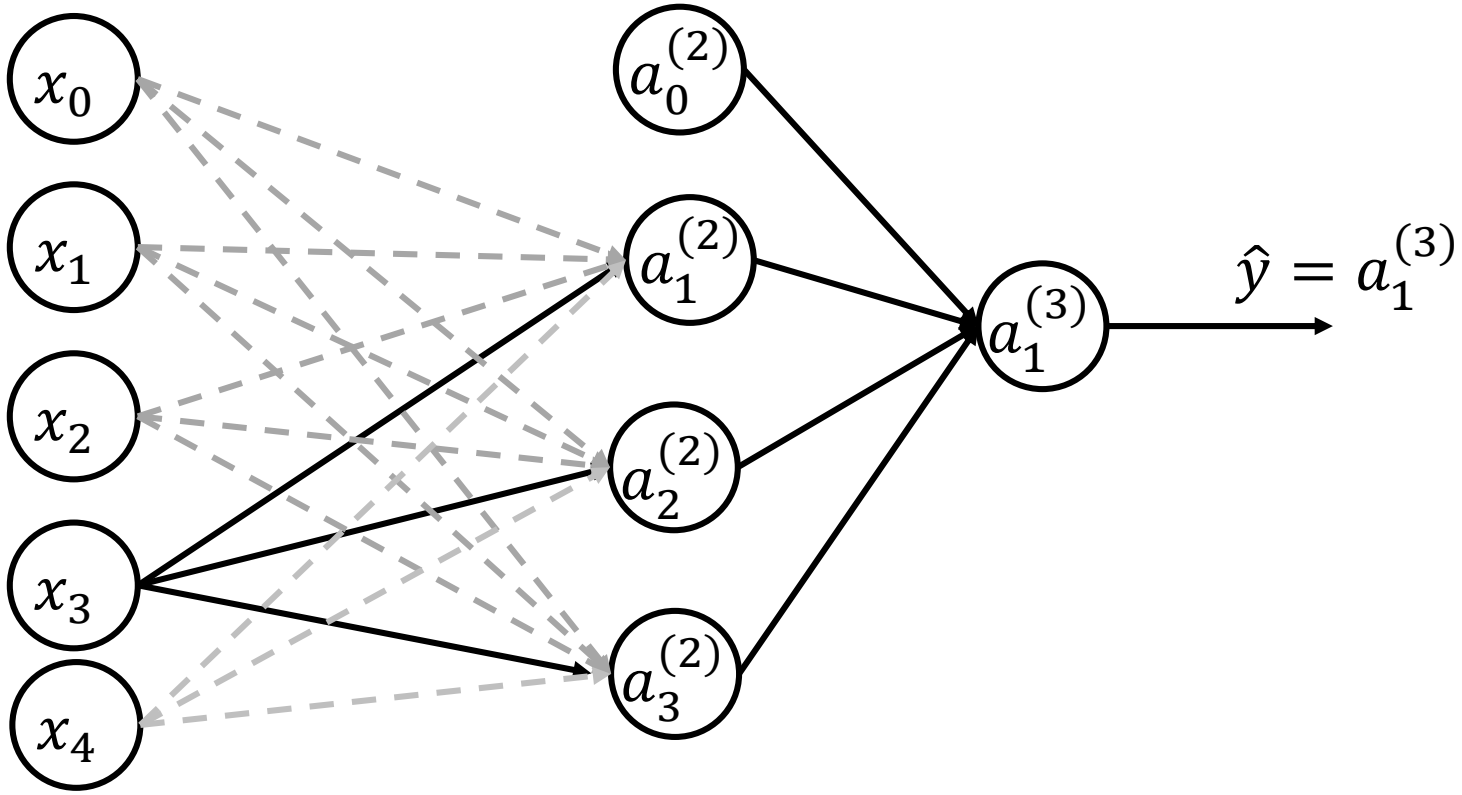
- The weights are initialized randomly and their values are illustrated in the table below.



Initial Weights	Value
$\theta_{12}^{(1)}$	0.4
$\theta_{22}^{(1)}$	0.7
$\theta_{32}^{(1)}$	0.2

Initial Weights

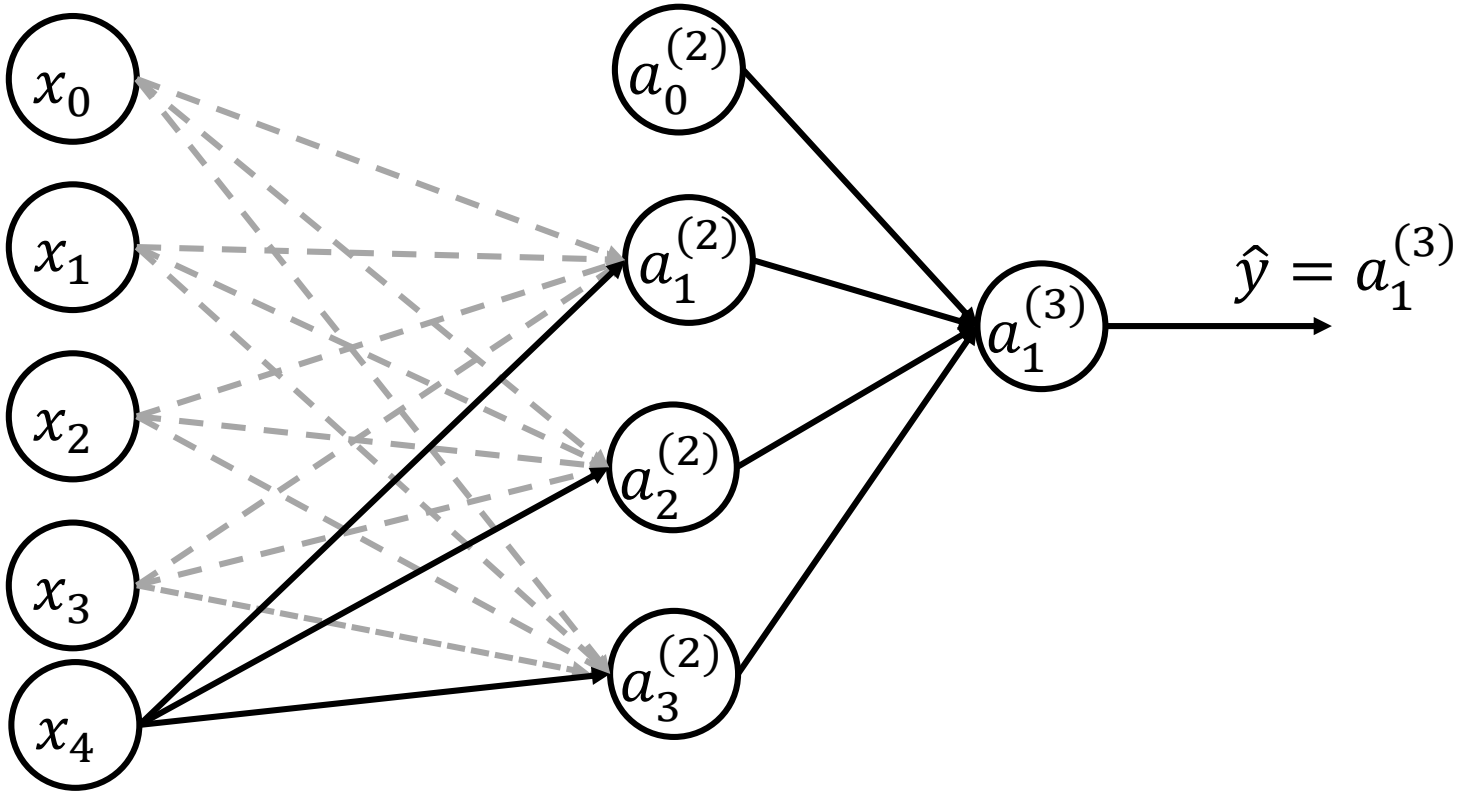
- The weights are initialized randomly and their values are illustrated in the table below.



Initial Weights	Value
$\theta_{13}^{(1)}$	0.4
$\theta_{23}^{(1)}$	-0.4
$\theta_{33}^{(1)}$	-0.1

Initial Weights

- The weights are initialized randomly and their values are illustrated in the table below.

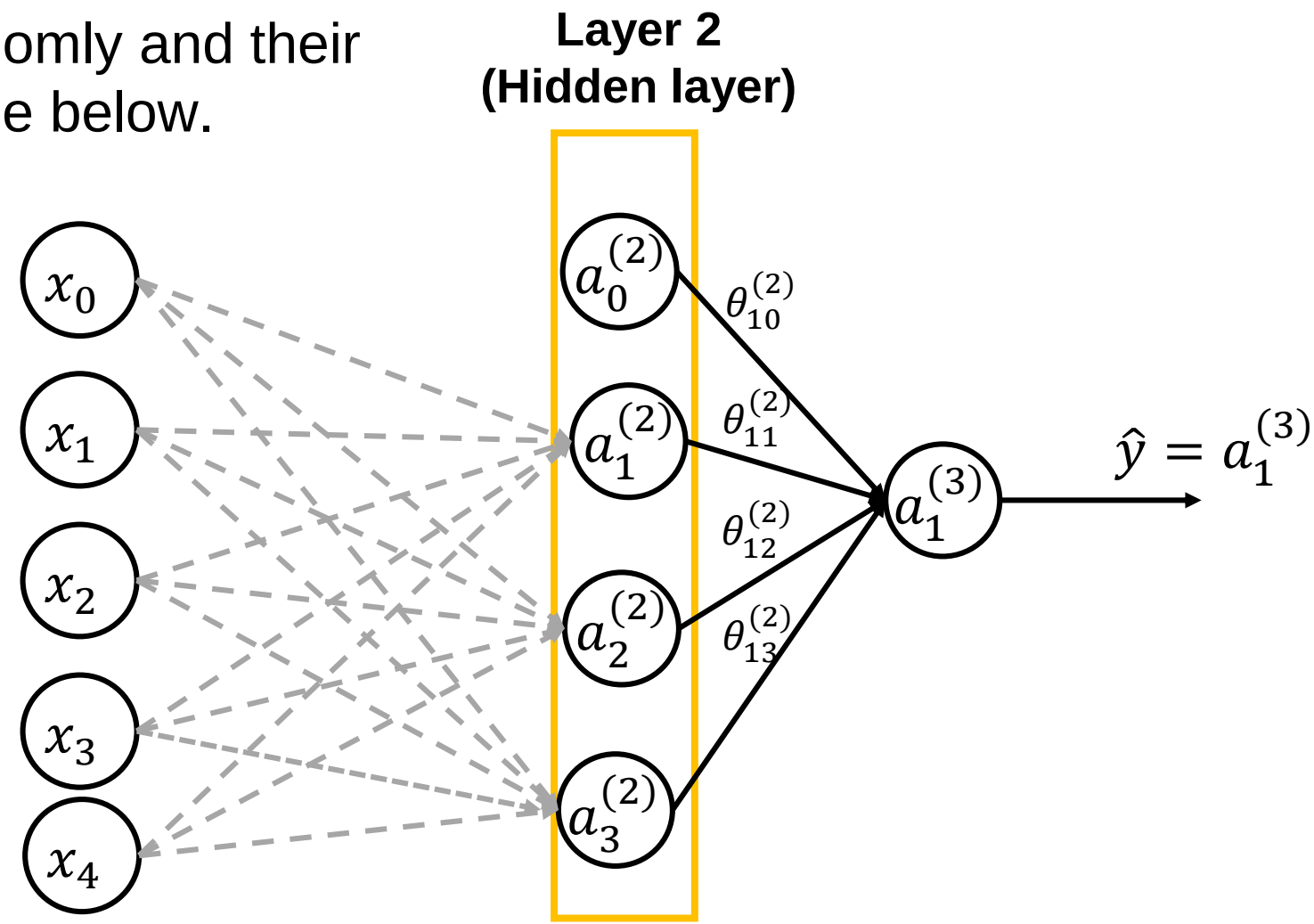


Initial Weights	Value
$\theta_{14}^{(1)}$	-0.7
$\theta_{24}^{(1)}$	-2.7
$\theta_{34}^{(1)}$	0.2

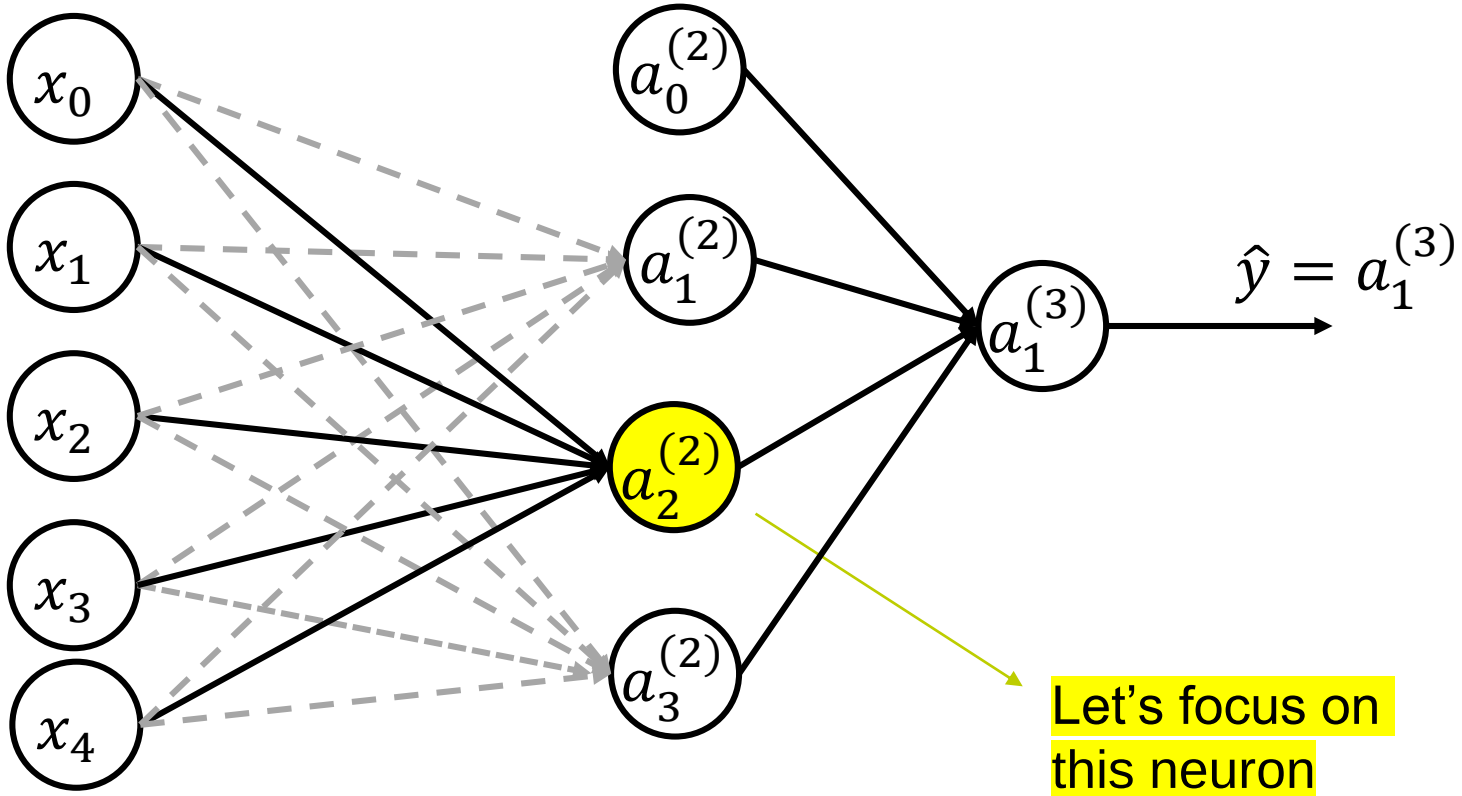
Hidden Layer

- The weights are initialized randomly and their values are illustrated in the table below.

Initial Weights	Value
$\theta_{10}^{(2)}$	0.1
$\theta_{11}^{(2)}$	0.1
$\theta_{12}^{(2)}$	0.3
$\theta_{13}^{(2)}$	1

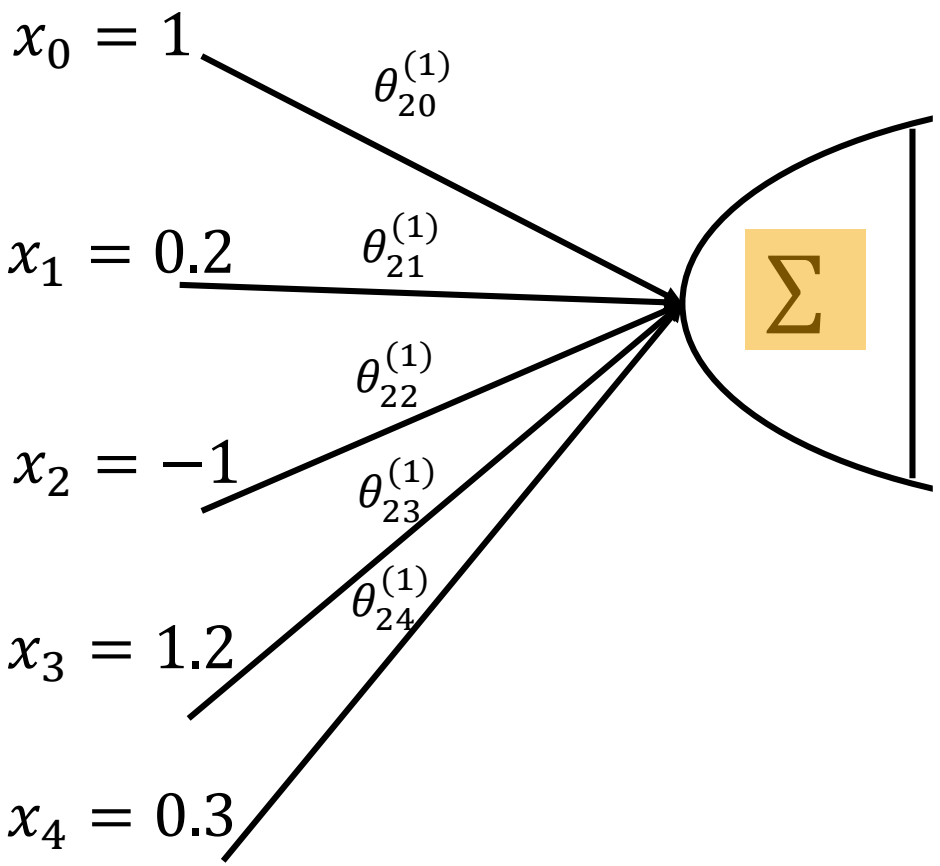


Neuron Activation Calculation



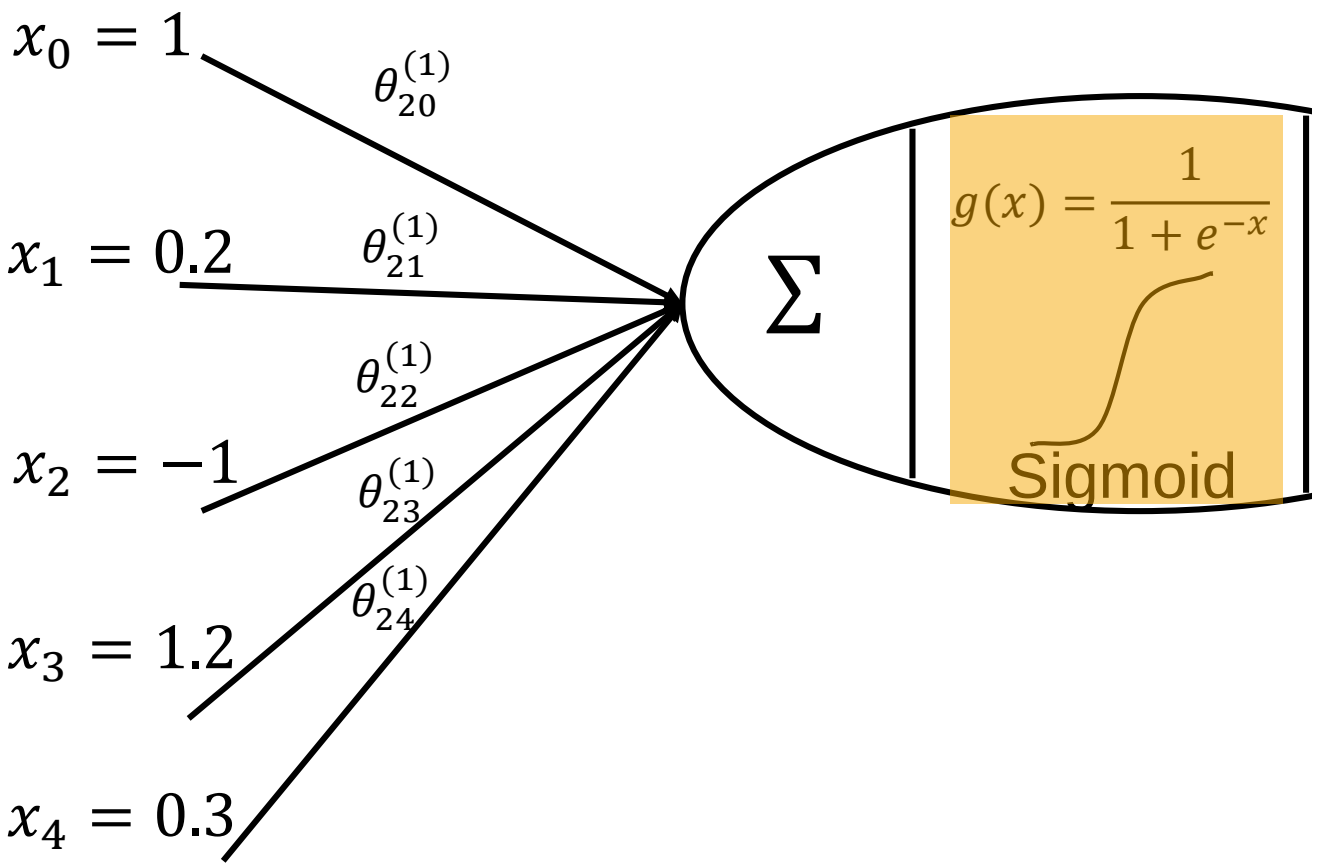
Mathematical model for the neuron $a_2^{(2)}$

- $a_2^{(2)} = g\left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 + \theta_{24}^{(1)} x_4\right) = g(z_2^{(2)})$



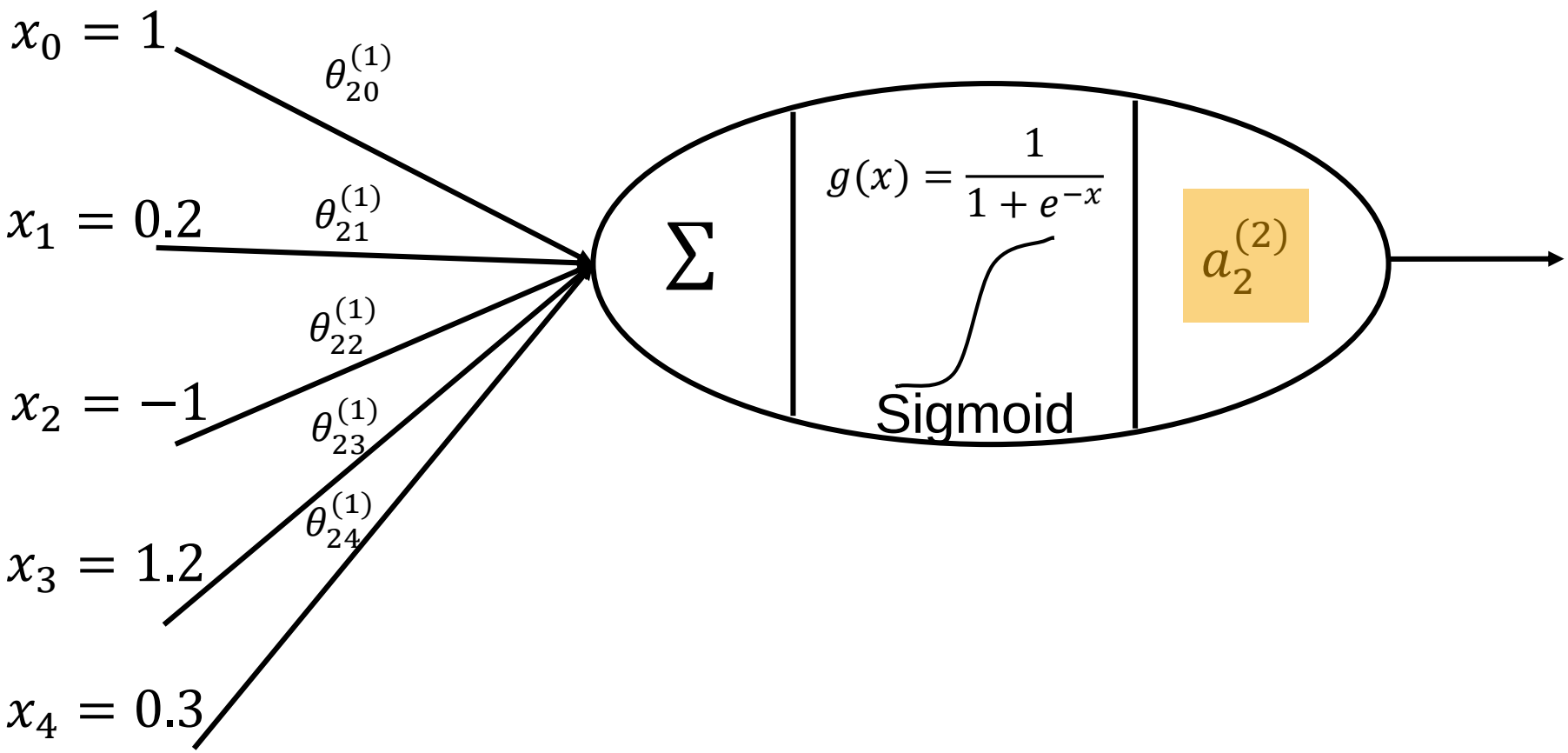
Mathematical model for the neuron $a_2^{(2)}$ - Activation function

- $a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 + \theta_{24}^{(1)} x_4) = g(z_2^{(2)})$



Mathematical model for the neuron $a_2^{(2)}$

- $a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 + \theta_{24}^{(1)} x_4) = g(z_2^{(2)})$



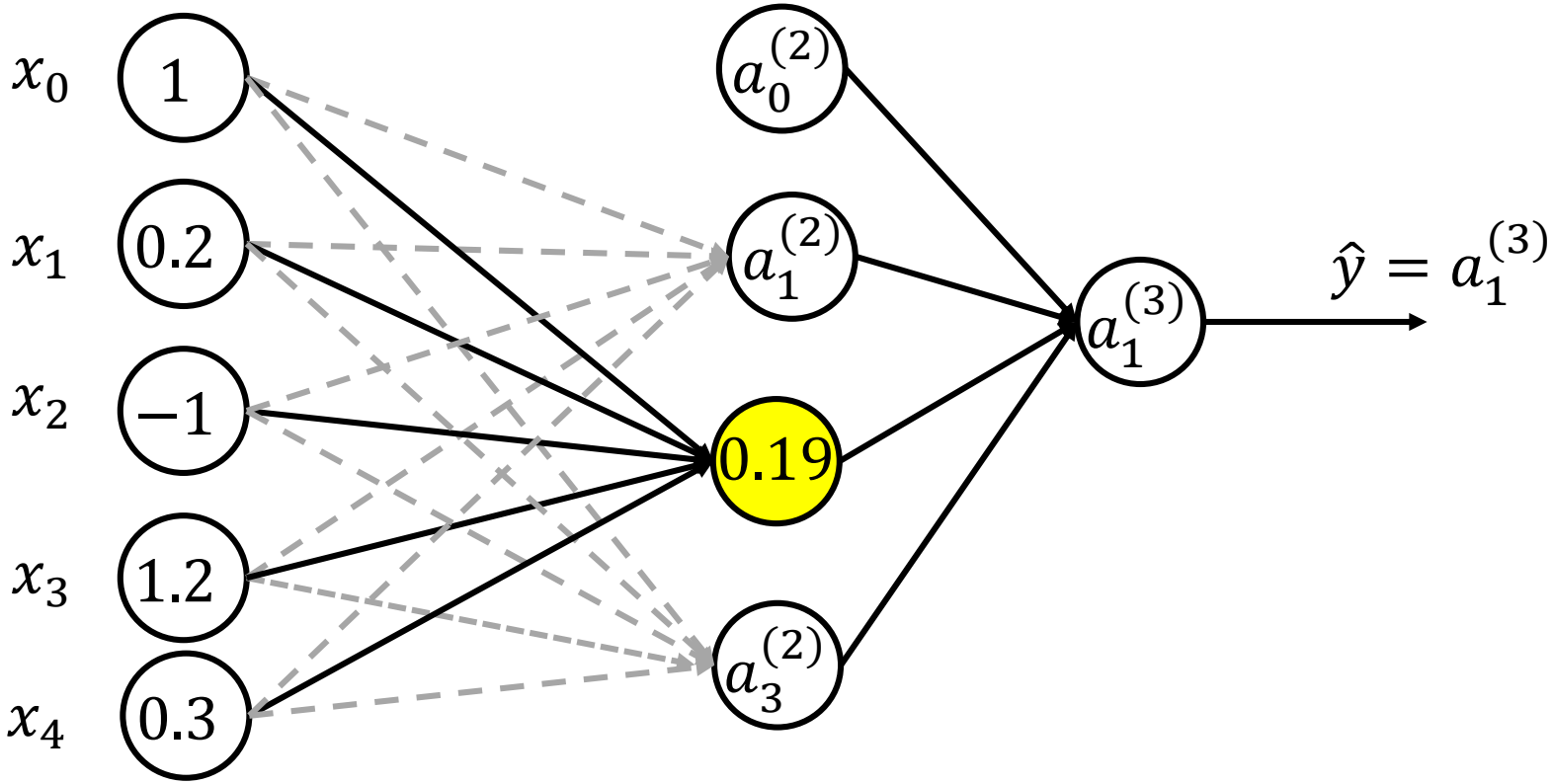
Calculation for the second neuron in hidden layer $a_2^{(2)}$

$$a_2^{(2)} = g\left(\theta_{20}^{(1)}x_0 + \theta_{21}^{(1)}x_1 + \theta_{22}^{(1)}x_2 + \theta_{23}^{(1)}x_3 + \theta_{24}^{(1)}x_4\right)$$

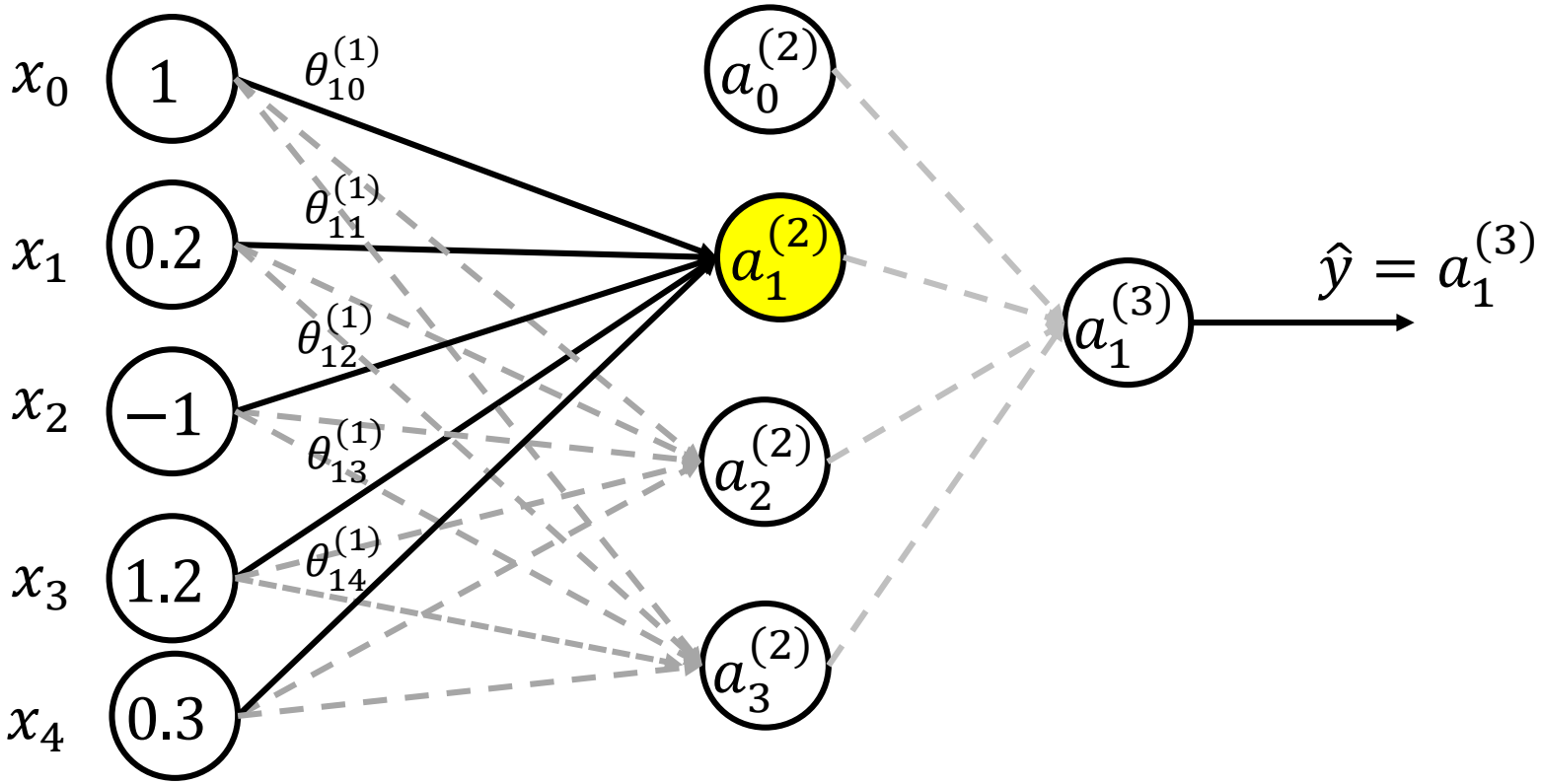
$$= g(0.3 \times 1 + 1.2 \times 0.2 + 0.7 \times (-1) + (-0.4) \times 1.2 + (-2.7) \times 0.3)$$

$$= g(-1.45) = \frac{1}{1 + \mathrm{e}^{+1.45}} = 0.19$$

Updated value for second neuron in hidden layer



Updated value for first neuron in hidden layer



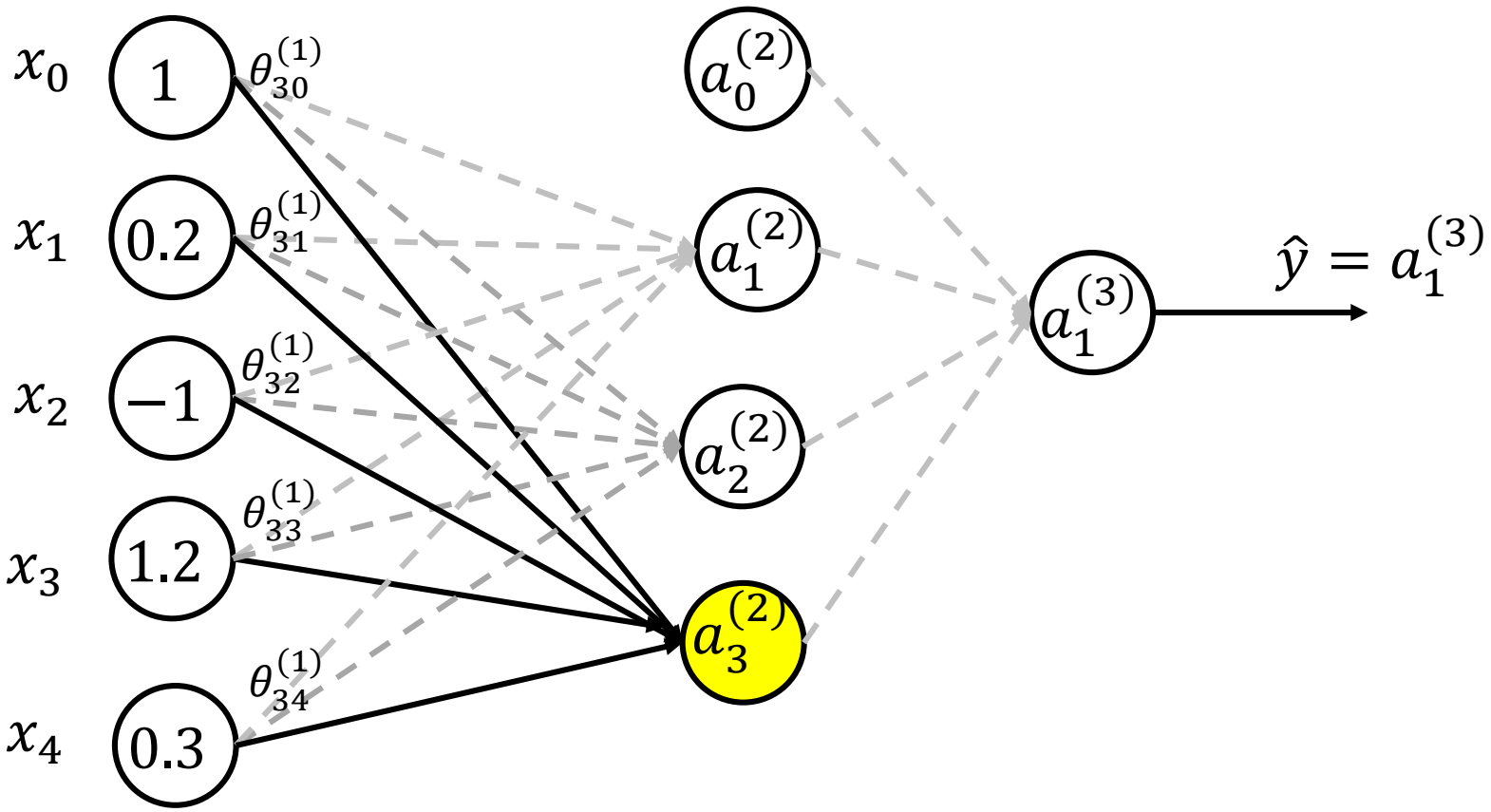
Calculation for the first neuron in hidden layer $a_1^{(2)}$

$$a_1^{(2)} = g\left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 + \theta_{14}^{(1)} x_4\right)$$

$$= g(1.0 \times 1 + 0.2 \times 0.2 + 0.4 \times (-1) + 0.4 \times 1.2 + (-0.7) \times 0.3)$$

$$= g(0.91) = \frac{1}{1 + e^{-0.91}} = 0.713$$

Calculation for third neuron in hidden layer



Calculation for the third neuron in hidden layer $a_3^{(2)}$

$$a_3^{(2)} = g\left(\theta_{30}^{(1)}x_0 + \theta_{31}^{(1)}x_1 + \theta_{32}^{(1)}x_2 + \theta_{33}^{(1)}x_3 + \theta_{34}^{(1)}x_4\right)$$

$$= g(0.4 \times 1 + 0.1 \times 0.2 + 0.2 \times (-1) + (-0.1) \times 1.2 + 0.2 \times 0.3)$$

$$= g(0.16) = \frac{1}{1 + e^{-0.16}} = 0.5399$$

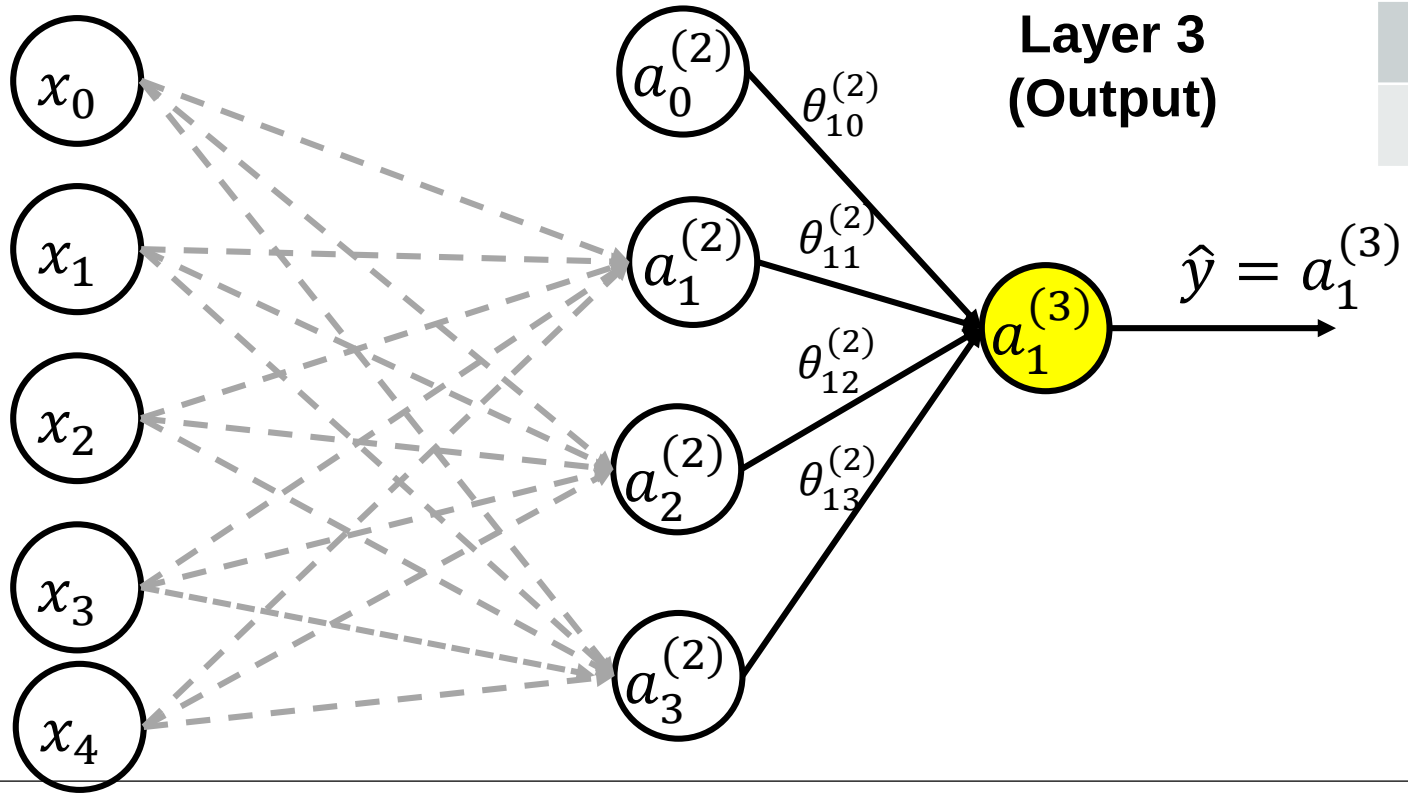
Output layer

$$\hat{y} = a_1^{(3)} = g\left(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)}\right) =$$

$$= g(0.1 \times 1 + 0.1 \times 0.713 + 0.3 \times 0.19 + 1 \times 0.539)$$

$$= 0.683$$

Initial Weights	Value
$\theta_{10}^{(2)}$	0.1
$\theta_{11}^{(2)}$	0.1
$\theta_{12}^{(2)}$	0.3
$\theta_{13}^{(2)}$	1

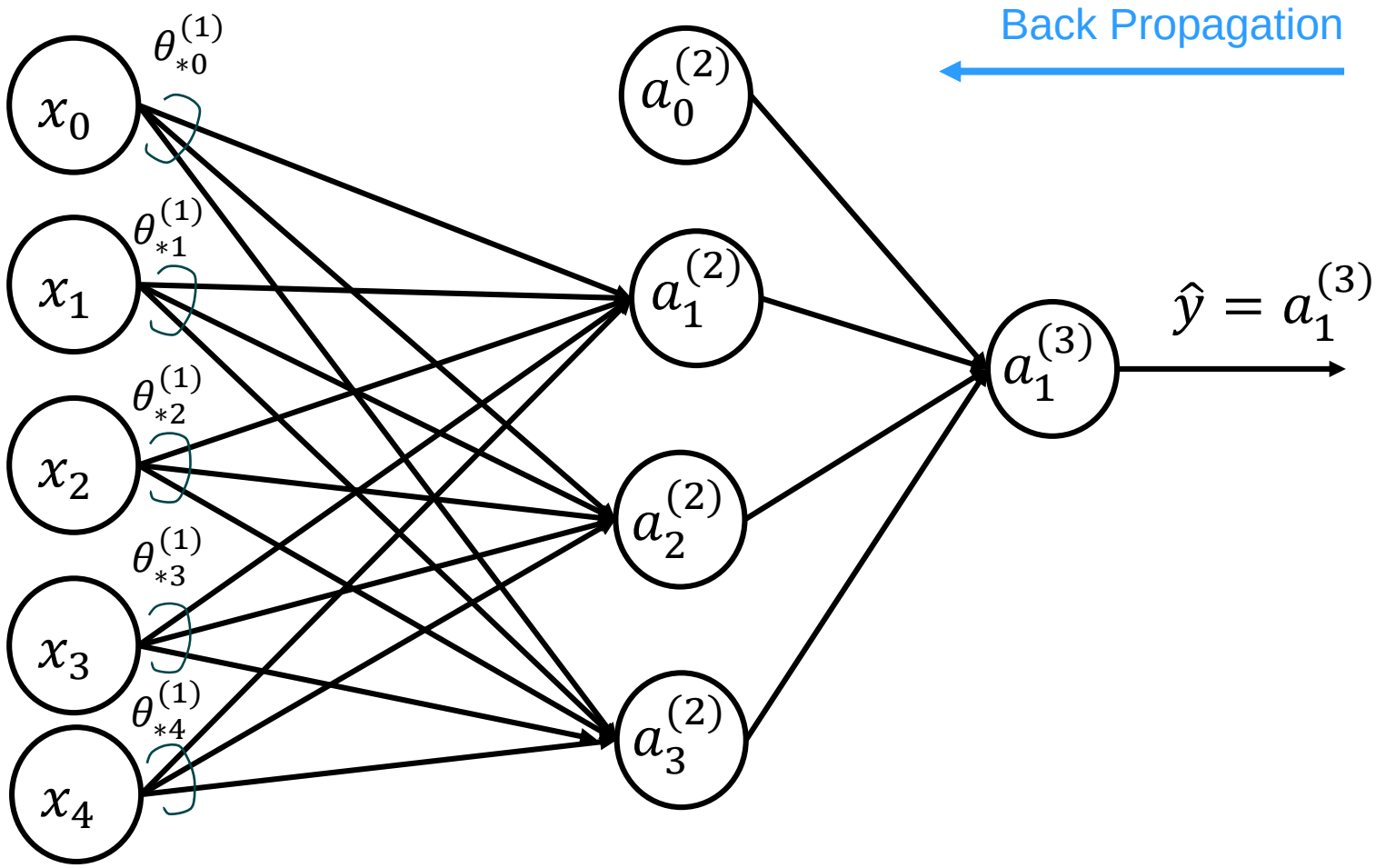


Back Propagation

Back propagation: target output is $y = 1$

Initial Weights	Value
$\theta_{10}^{(1)}$	1.0
$\theta_{20}^{(1)}$	0.3
$\theta_{30}^{(1)}$	0.4
$\theta_{11}^{(1)}$	0.2
$\theta_{21}^{(1)}$	1.2
$\theta_{31}^{(1)}$	0.1
$\theta_{12}^{(1)}$	0.4
$\theta_{22}^{(1)}$	0.7
$\theta_{32}^{(1)}$	0.2
$\theta_{13}^{(1)}$	0.4
$\theta_{23}^{(1)}$	-0.4
$\theta_{33}^{(1)}$	-0.1
$\theta_{14}^{(1)}$	-0.7
$\theta_{24}^{(1)}$	-2.7
$\theta_{34}^{(1)}$	0.2

$\theta_{*0}^{(1)}$ $\theta_{*1}^{(1)}$ $\theta_{*2}^{(1)}$ $\theta_{*3}^{(1)}$ $\theta_{*4}^{(1)}$



Backpropagation (BP) Algorithm

- For output unit (L=3) we compute error term $\boldsymbol{\delta}$:

$$\boldsymbol{\delta}^{(3)} = \mathbf{a}^{(3)} - \mathbf{y}$$

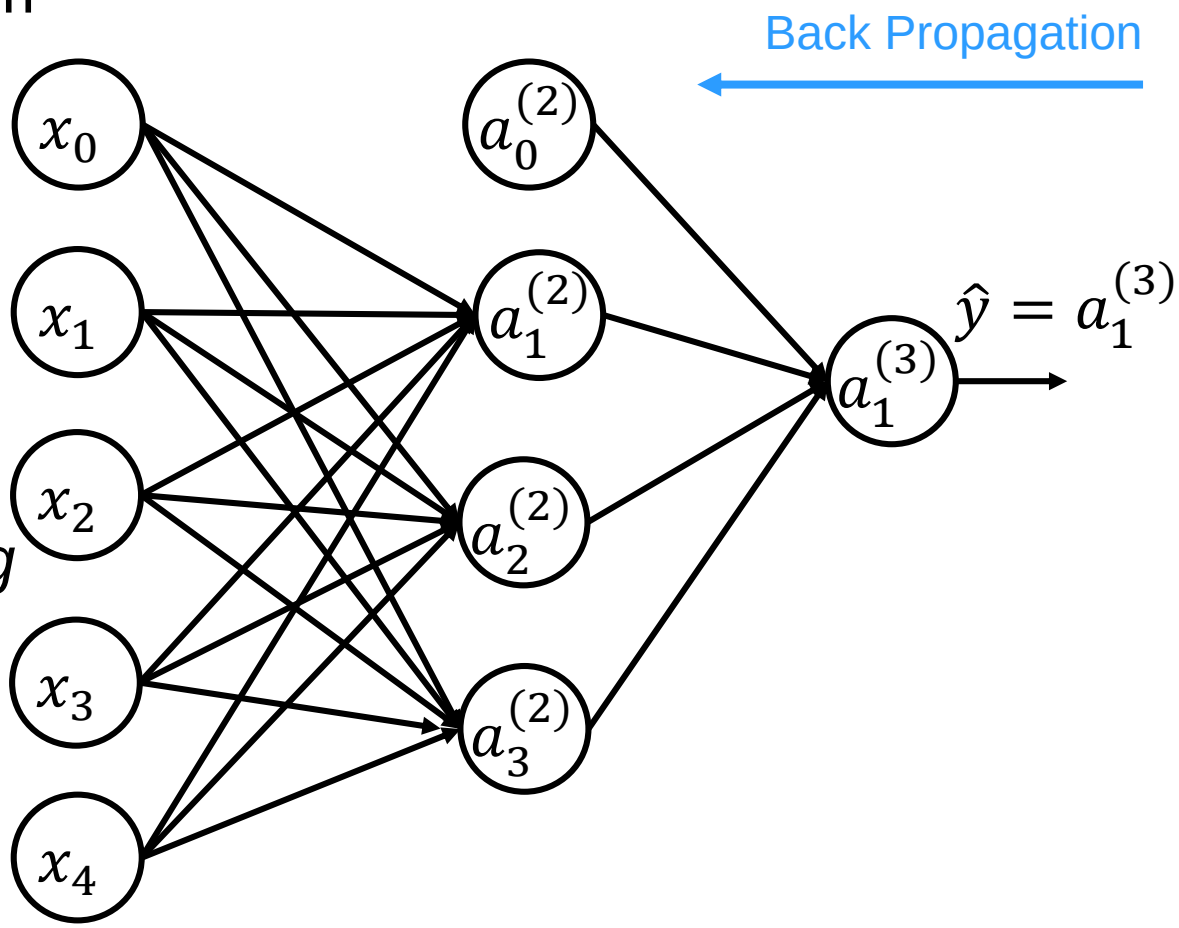
- For layer2, we compute $\boldsymbol{\delta}$ term:

$$\boldsymbol{\delta}^{(2)} = (\boldsymbol{\theta}^{(2)})^T \boldsymbol{\delta}^{(3)} \odot g'(\boldsymbol{\theta}^{(1)} \mathbf{a}^{(1)})$$

- $g'(\mathbf{z}^{(l)})$ is derivative of activation function g at the input value $\mathbf{z}^{(l)}$:

$$g'(\mathbf{z}^{(l)}) = \mathbf{a}^{(l)} \odot (1 - \mathbf{a}^{(l)})$$

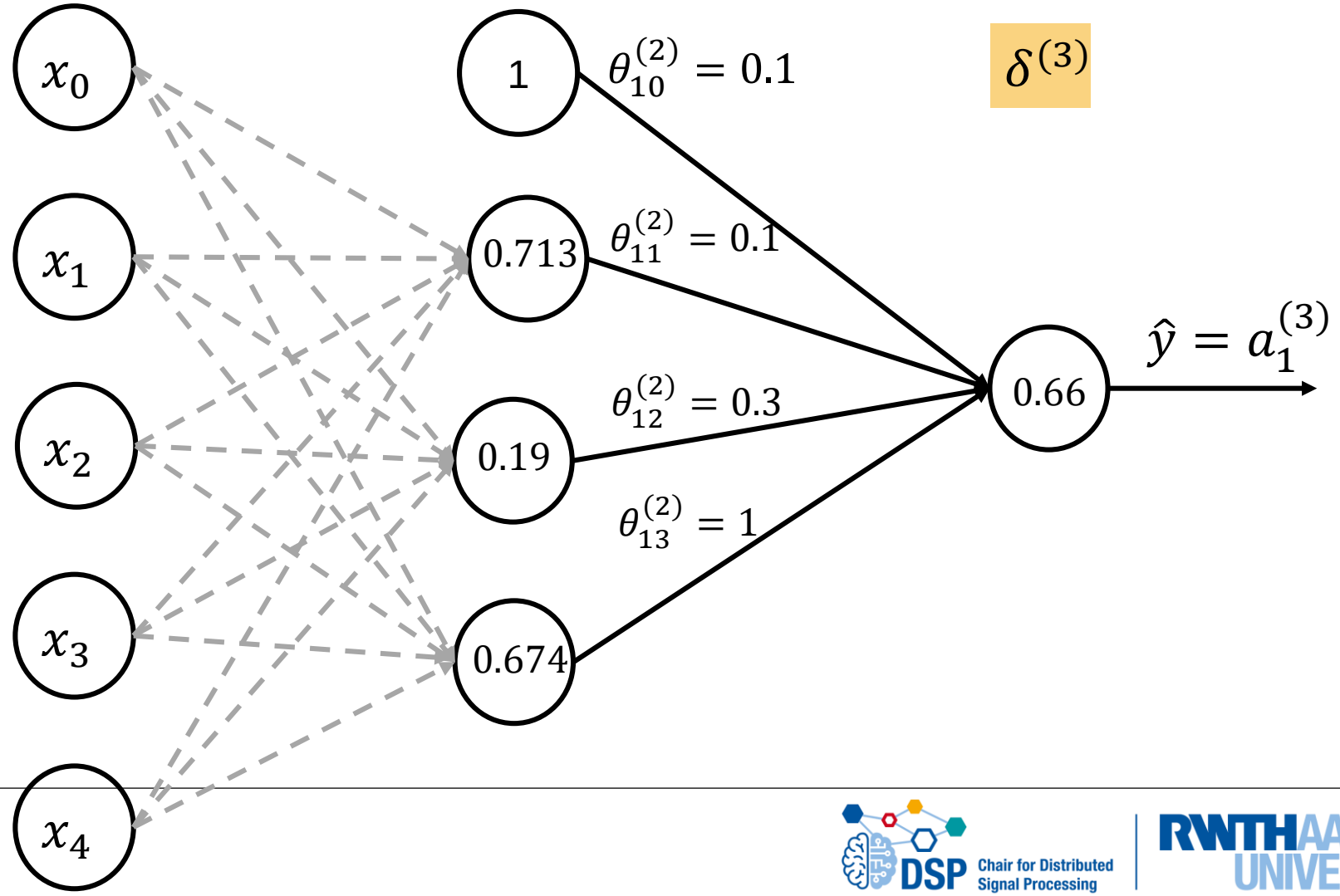
- Here, we assume the activation function for hidden layer is Sigmoid ($g(x) = \frac{1}{1+e^{-x}}$)



Error computation for L=3

- We compute error for layer 3:

$$\delta^{(3)} = a_1^{(3)} - y$$
$$= 0.682 - 1$$
$$= -0.318$$

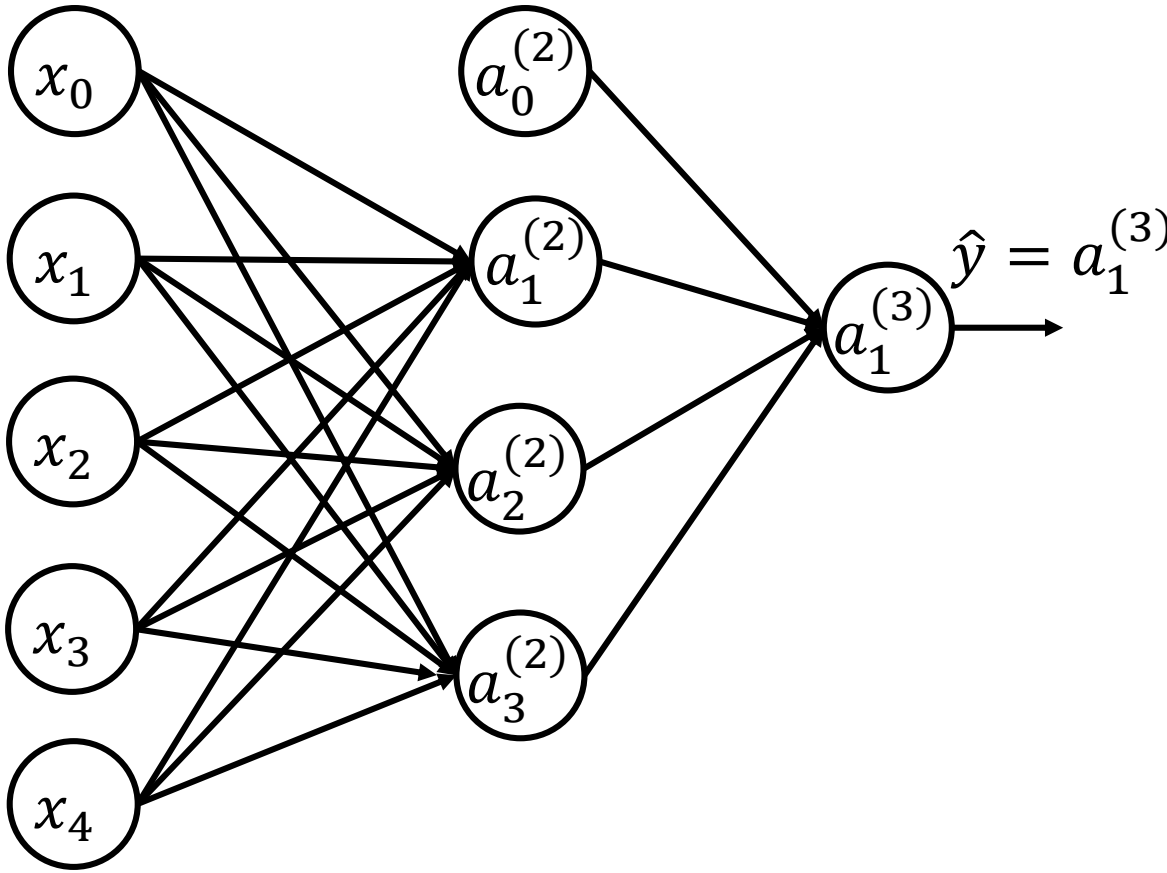


Error computation for L=2

- We compute error for layer 2:

$$\delta_1^{(2)} = \theta_{11}^{(2)} \delta_1^{(3)} a_1^{(2)} \left(1 - a_1^{(2)}\right) = -0.00651$$
$$\delta_2^{(2)} = \theta_{12}^{(2)} \delta_1^{(3)} a_2^{(2)} \left(1 - a_2^{(2)}\right) = -0.0146$$
$$\delta_3^{(2)} = \theta_{13}^{(2)} \delta_1^{(3)} a_3^{(2)} \left(1 - a_3^{(2)}\right) = -0.0789$$

$\delta^{(2)}$



Parameters update

- With all errors shown, now we have to update the parameters

- Parameters update rule is:

$$\boldsymbol{\theta}^{(l)} \leftarrow \boldsymbol{\theta}^{(l)} - \alpha \frac{\partial J(\theta)}{\partial \theta^{(l)}}$$

$$\boldsymbol{\theta}^{(l)} - \alpha \boldsymbol{\delta}^{(l+1)} \left(\mathbf{a}^{(l)}\right)^{T}$$

$$\boldsymbol{\theta}_0^{(l)} \leftarrow \boldsymbol{\theta}_0^{(l)} - \alpha \boldsymbol{\delta}^{(l+1)}$$

- where learning rate $\alpha = 0.01$

Parameters update L=2

$$\theta_{ji}^{(2)} \leftarrow \theta_{ji}^{(2)} - \alpha \delta_j^{(3)} a_i^{(2)}$$

$$\theta_{j0}^{(2)} \leftarrow \theta_{j0}^{(2)} - \alpha \delta^{(3)}$$

$$\theta_{11}^{(2)} \leftarrow 0.1 - 0.01 \times (-0.318) \times 0.713 = 0.1023$$

$$\theta_{12}^{(2)} \leftarrow 0.3 - 0.01 \times (-0.318) \times 0.19 = 0.30041$$

$$\theta_{13}^{(2)} \leftarrow 1 - 0.01 \times (-0.318) \times 0.539 = 1.0017$$

$$\theta_{10}^{(2)} \leftarrow 0.1 - 0.01 \times (-0.318) = 0.1031$$

Parameters update L=1

$$\theta_{11}^{(1)} \leftarrow 0.2 - 0.01 \times (-0.00651) \times 0.2 = 0.20001$$

$$\theta_{21}^{(1)} \leftarrow 1.2 - 0.01 \times (-0.0146) \times 0.2 = 1.200002$$

….

$$\theta_{34}^{(1)} \leftarrow 0.2 - 0.01 \times (-0.0789) \times 0.3 = 0.2002$$

$$\theta_{10}^{(1)} \leftarrow 1.0 - 0.01 \times (-0.00651) \times 0.2 = 1.00001$$

$$\theta_{20}^{(1)} \leftarrow 0.3 - 0.01 \times (-0.0146) \times 0.2 = 0.30002$$

$$\theta_{30}^{(1)} \leftarrow 0.4 - 0.01 \times (-0.0789) \times 0.2 = 0.40015$$

- All values are calculated and provided in the following table.

Parameters update L=1

- We calculate the updated value of parameters.

Parameter	Updated value	Parameter	Updated value
$\theta_{11}^{(1)}$	0.20001	$\theta_{13}^{(1)}$	0.400001
$\theta_{21}^{(1)}$	1.200002	$\theta_{23}^{(1)}$	-0.39992
$\theta_{31}^{(1)}$	0.100001	$\theta_{33}^{(1)}$	-0.09992
$\theta_{12}^{(1)}$	0.400002	$\theta_{14}^{(1)}$	-0.699992
$\theta_{22}^{(1)}$	0.699854	$\theta_{24}^{(1)}$	-2.699992
$\theta_{32}^{(1)}$	1.99921	$\theta_{34}^{(1)}$	0.2002
$\theta_{10}^{(1)}$	1.00001		
$\theta_{20}^{(1)}$	0.30002		
$\theta_{30}^{(1)}$	0.40015		

Convergence Check

- Once you update all parameters, do the next round of forward propagation
- Then perform again back propagation and parameters update.
- Repeat this loop till cost value improvement is lower than a threshold.

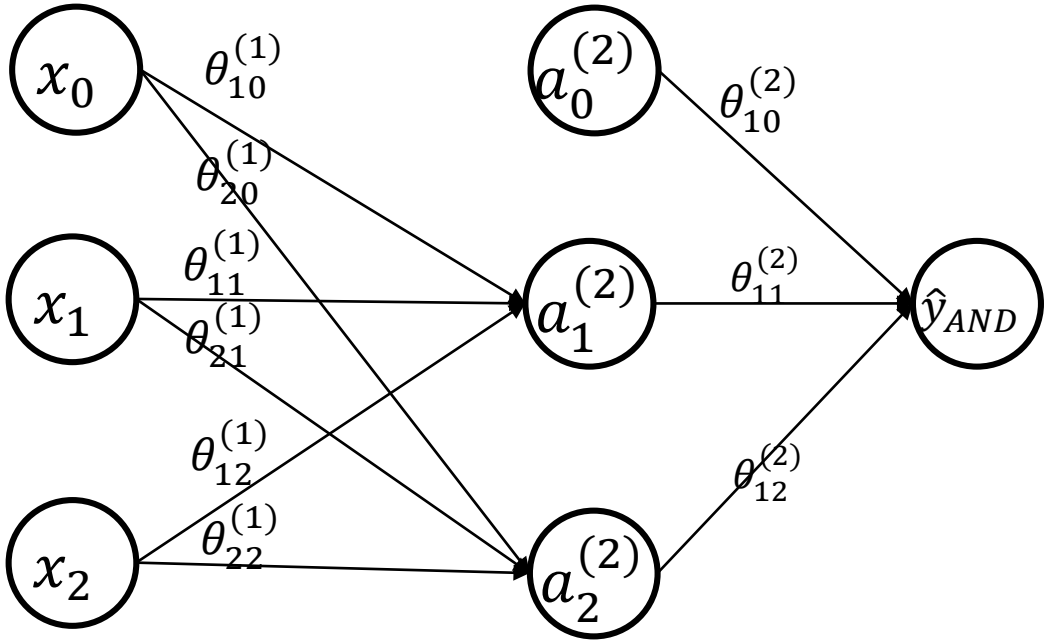
Assignment

Neural network for AND and XOR logic gates

- **Objective:** You will implement a small fully connected neural network in Python and train it to learn two logic gates:
  - AND
  - XOR
- Use the same neural network architecture for both tasks and train two separate models.

Neural network architecture

- Consider the neural network structure as:
  - 2 input neurons
  - 1 hidden layer with 2 neurons
  - 1 output neuron
  - Sigmoid activation in both the hidden layer and the output layer: $g(x) = \frac{1}{1+e^{-x}}$



Dataset for training

- Use the following 4 input pairs as your dataset for training.
  - There is no separate test dataset in this assignment.

x_1	x_2	y_{and}	y_{xor}
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Cost function and training method

- Consider the soft-max cost function as

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - \left(h_{\boldsymbol{\theta}}\left(\boldsymbol{x}^{(i)}\right)\right)\right)\right]$$

m is the number of training samples.

Tasks

- **Assumptions:** Train the network using gradient descent. Assume learning rate is $\alpha = 0.1$ and number of epochs is 10000.
  - Random initialization of parameters for all neurons and bias is given.

Task1. Implement the sigmoid activation function

- The corresponding function is *sigmoid(x)*

Task2. Implement the cost function $J(\boldsymbol{\theta})$

- To avoid $\log(0)$, use a small value like 1e-15
- The corresponding function is *cost(y_true, y_pred)*

Tasks

Task3. Implement the forward propagation

- For each input pair (x_1, x_2) return the predicted output
- A python dictionary called "*saved*" containing the intermediate values is needed for backpropagation.
- The corresponding function is *forward(input1, input2, p)*
- p is a python dictionary storing neural network's parameters, consisting of weights and bias values.

Task4. Implement the backpropagation

- Using the "*saved*" values from the forward propagation, compute the gradients of the cost with respect to all weights and biases.
- Return a python dictionary called "*grads*" with the same keys as p
- The corresponding function is *backward(y_true, saved, p)*

Tasks

Task5. Implement the parameter update step

- Update the parameters using the average gradient over all training samples
- The corresponding function is *apply_updates(p, sum_grads, learning_rate, n_samples)*
- *sum_grads* is a python dictionary which sums gradients over all samples

Submission

- Submit the complete **.py** file containing:
  - The full training implementation
  - Clear and inline comments explaining your code

If you experience any issues uploading the $.py$ files directly, compress it into a .zip or .tar file and upload the compressed file instead

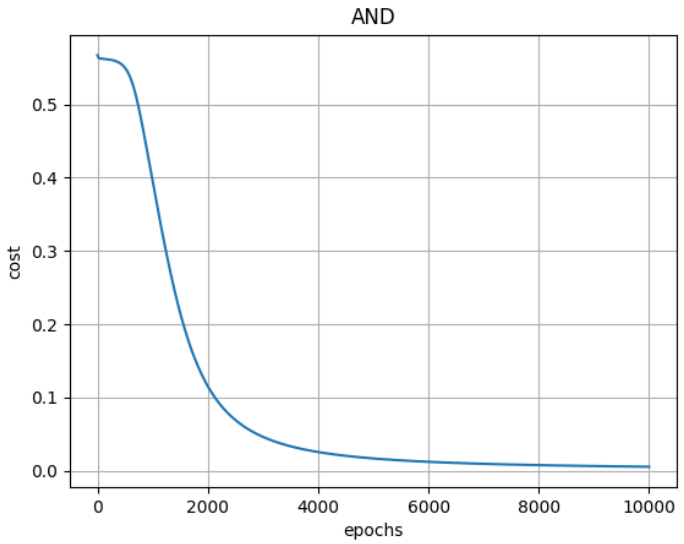
- Submit your code on Moodle by **June 8, 2026, at 10:30 AM**.
- Do not import any additional libraries or modules that are not already included in the provided code.

❖ If the code fails to run, it will receive no points.

Expected result

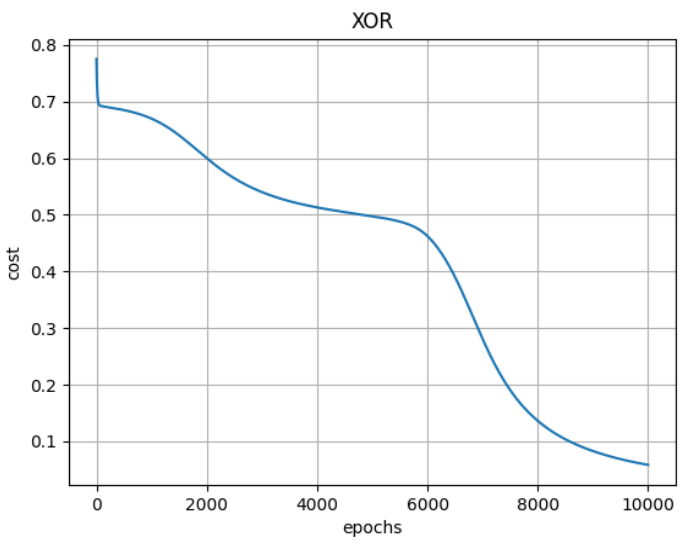
AND result

```
Testing AND Gate Neural Network
Input: [0 0], Predicted: [False], Actual: [0]
Input: [0 1], Predicted: [False], Actual: [0]
Input: [1 0], Predicted: [False], Actual: [0]
Input: [1 1], Predicted: [ True], Actual: [1]
```



XOR result

```
Testing XOR Gate Neural Network
Input: [0 0], Predicted: [False], Actual: [0]
Input: [0 1], Predicted: [ True], Actual: [1]
Input: [1 0], Predicted: [ True], Actual: [1]
Input: [1 1], Predicted: [False], Actual: [0]
```



End of exercise 5